

SO-101 Dinner Table: a scored bimanual benchmark

Bimanual VLA Manipulation with Multi-Modal Reasoning — Setting Up a Dinner Table

AI Infra Summit Hackathon · Intel Physical AI Online Challenge · lablab.ai

What it is

- A randomized dual-SO-101 dinner-table task in MuJoCo
- Five ordered success predicates, scored per seed
- A scorer controlled both ways, so its number can be falsified

What it measured

19 / 50 sub-goals over 10 seeds

0 / 50 with the arms held in the home pose · task success 0 / 10



The problem this entry actually attacks

A bimanual demo video is easy to make and impossible to check.

- Did the result survive a new random seed, or was it one lucky run?
- Would the scorer have fired anyway with the arms held still?
- Which of the five steps in the instruction actually happened?
- Was it one arm doing everything, with the second along for the ride?

Who has this problem

Teams training manipulation policies for the low-cost open arms this track names report success rates on environments nobody else can rerun. So this entry built the measuring instrument first, and reports what the instrument says — including the zeros.

What is built

SCENE `envs/dinner_table.py`

Two SO-101 arms — five positioning joints and a parallel jaw each

A cabinet with a prismatic drawer holding the fork and the spoon

Plate, mug and bottle on the table

`nq=48 nu=12 nbody=25 ngeom=133
nmesh=36`

RANDOMIZER `envs/randomize.py`

Object geometry, mass, friction, lighting, background and placement vary per seed

Rejection-sampled: every episode is reachable and never blocks the drawer

`plate 94-115 mm across`

SCORER `envs/task.py`

Five ordered sub-goals: drawer, fork, spoon, plate, mug

45-50 mm placement tolerance, 25° upright check

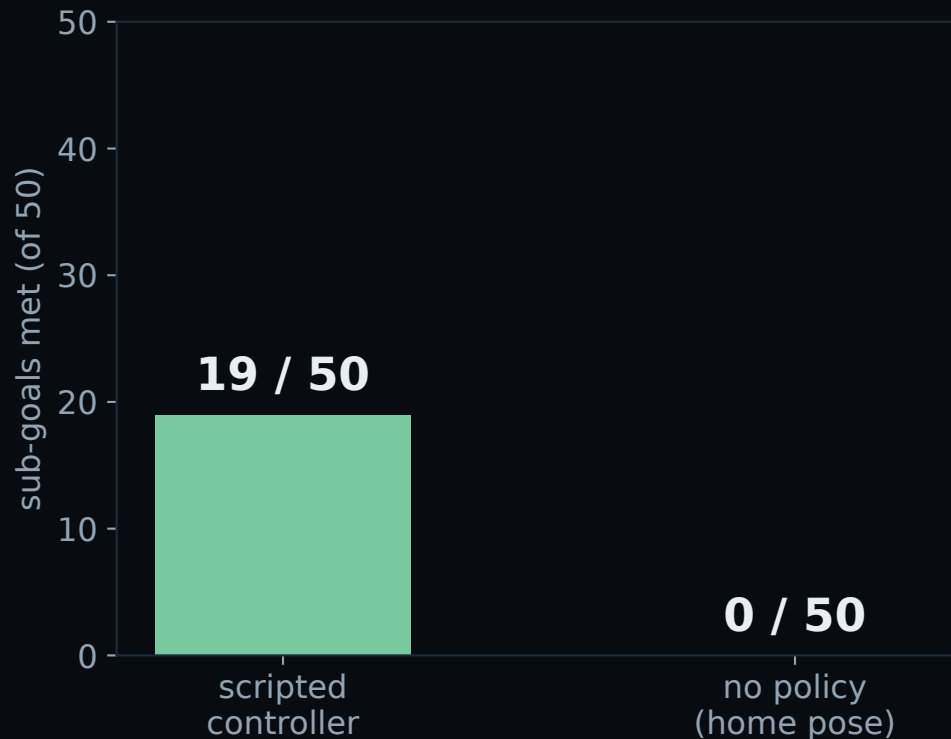
Also logs order, arm, hand-off and drops

`5 predicates · 10 seeds`

The instruction, verbatim from `envs/task.py`:

Open the top drawer, pick up the fork and the spoon and lay them either side of the place setting, put the plate on the mat, then set the mug down to the right of the plate.

The number ships with its own falsifier



Same seeds. Same scorer bytes. Only the controller differs.

Any sub-goal that fired with the arms held still would mean the scorer produced the result, not the controller. None fires.

Controls that pass, both directions

16 / 16 scene checks	structure and physics
11 / 11 scorer controls	accept and reject
18 / 18 perception + OpenVINO controls	accept and reject

A scorer that can only say yes proves nothing. Every predicate is driven from both sides: a state that must score, and a state that must not. Both are in the repository and both run in seconds.

RESULTS

Results, per sub-goal — including the zeros



What the run does not show

task success 0 / 10

in-order prefix 2 of 5 on every seed

object hand-offs 7 / 10

objects dropped 0

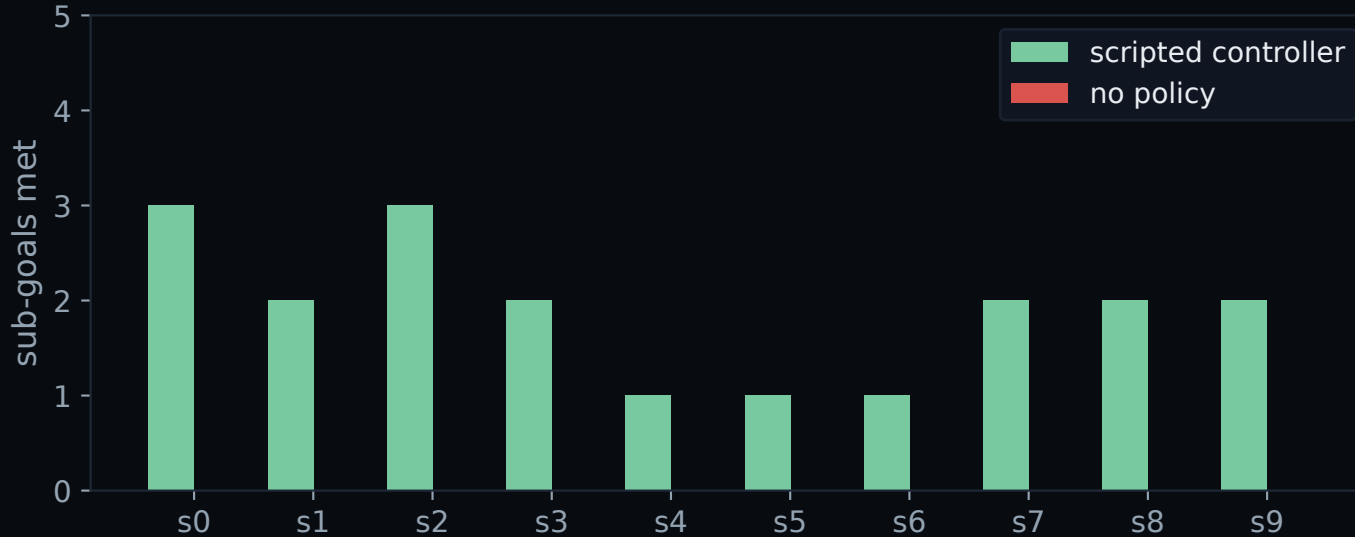
both arms touched an object on 10 / 10 seeds

all 10 recorded hand-offs are the two arms taking the drawer handle in turn, not an object

The plate is hooked by its rim and dragged flat, not picked and placed: it is 94–115 mm across the ten evaluation seeds, wider than the jaw can both clear and close on. The scorer does not ask how the plate reached the mat. This deck does.

Worst IK residual over the ten rollouts: 294.8 mm across 1,648 planned moves — waypoints the arm never reaches are the open defect this entry has not fixed.

Robustness: one controller, ten random seeds



What varies per seed

plate diameter 94–115 mm

mug height 28–40 mm

cutlery length 36–43 mm

mass $\times 0.60$ –1.60

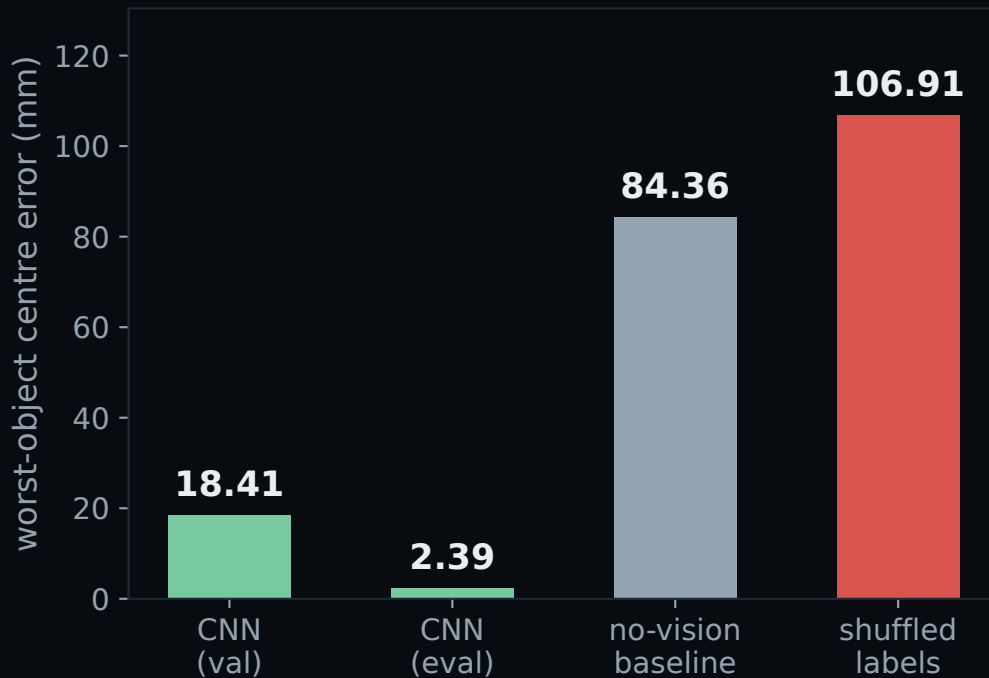
friction $\times 0.60$ –1.29

light diffuse 0.60–0.94

Robustness is evidenced for the two steps that fire and for nothing else: the drawer on 10 / 10 seeds and the plate on 5 / 10. There is still no performance on the other three sub-goals to be robust about. Placement randomization is rejection-sampled, so a failed episode is a controller failure and never an impossible scene.

The demo video covers the same 10 seeds and is captioned frame by frame from the live predicate state, so it cannot claim more than the evaluation reports · sha256 19bcc5bf4cd3...

Application of technology — perception



One overhead frame in, table layout out

- 520,768 parameters, spatial softmax head
- 6,990 training frames, 840 validation, 10 evaluation
- 80 epochs; the three splits are seed-disjoint by construction
- 18.41 mm validation, 2.39 mm on the ten evaluation seeds

Two controls it has to beat

no-vision baseline 84.36 mm · shuffled labels 106.91 mm

Stated plainly: this is perception, not a policy. It outputs scene state, not actions. It IS in the control loop, and it costs 10 sub-goals to put it there: 9 / 50 against 19 / 50.

It answers one of the VLA criterion's four demands. Natural language, multi-step task context and re-planning are still absent.

Application of technology — OpenVINO

Exported IR, and what each precision costs

precision	weights	eval error
FP32	2,031 KiB	2.44 mm
FP16	1,015 KiB	2.29 mm
INT8	515 KiB	12.51 mm

INT8 is 3.9× smaller than FP32 and costs 10.07 mm

What the bench reports

- single-stream latency, p50 through p99
- async throughput at the plugin's own optimal request count
- the execution precision the plugin actually chose
- the model's task quality in millimetres
- 9 device/precision rows on this host

On this host, OpenVINO 2026.3.1: CPU/FP32 0.426 ms p50, 5,488 fps async.

The cost of INT8 is reported and not hidden — a control named `reject_int8_is_free` fails if INT8 is ever recorded as no worse than FP32.

Those latency numbers are not the measurement this track scores. The next slide says why, and what would fix it.

No Intel Core Ultra measurement exists here

The track asks for: Intel Core Ultra Series 2/3 with OpenVINO

What this box is

AMD EPYC 9654 96-Core Processor

two NVIDIA L40S, no Intel silicon, no NPU

NOT_THE_REQUIRED_MEASUREMENT

What closes it

one unchanged run of

`python3 scripts/bench_openvino.py`

on a Core Ultra Series 2/3 machine, NPU included

The accept path is tested without owning the hardware: the verdict function accepts 3 real Core Ultra part strings and rejects 4 non-Core-Ultra ones, including this host, a Xeon and a Core i9. So the branch that would report success is exercised, not merely the branch that refuses.

AMD numbers are not offered under an Intel heading. The report stamps itself, in the file, as not the required measurement — because a judge who cannot trust the small claims cannot check the large ones.

What is not built

This entry is smaller than the track asks for, and exact about which parts are missing.

- The learned ACT policy loses to the script: 3 / 50 against 19 / 50.
- No VLA, no language conditioning: the instruction is a fixed string.
- Task success 0 / 10 — the table has never been set.
- spoon_placed 0 / 10.
- Object hand-off only ever with the mug, on 7 / 10 seeds.
- Perception in the loop costs 10 sub-goals (9 / 50 against 19 / 50) and estimates no height, yaw, dimensions, fork or spoon.
- No Intel Core Ultra measurement: NOT_THE_REQUIRED_MEASUREMENT.
- The plate is dragged by its rim, not carried.

Every one of these is also stated in the repository README, in TECHNICAL_SUMMARY.md section 8 and in the submission text — and scripts/test_slides.py fails if this slide stops stating any of them.

Value: who it is for, and what would pay for it

The user

Robotics teams training manipulation policies for low-cost open arms.
Today they report success rates on environments nobody else can rerun.

The offer

The instrument, not the policy: a seeded, randomized, ordered-sub-goal task with a falsifier attached — cheap enough to run on a laptop.

The route to revenue

A versioned open benchmark with hosted evaluation, so a team can quote a number it can defend. Sponsorship and private leaderboards follow that, not the other way round.

Market size is not estimated here, because nothing was measured that would support an estimate. None of the route above is built: there are no users and there is no revenue. It is the route, not a claim.

What exists today that another team could use: the scene, the randomizer, the scorer, the two-way controls and the evaluation script.

Future goals and plans

1

Perception into the control loop

closes the privileged-state gap; the model already reads the layout to 2.39 mm on the evaluation seeds

2

A language-conditioned policy

trained on scripted-controller rollouts — the missing VLA the track scores

3

A grasp the plate survives

a 94–115 mm plate against a jaw that cannot both clear it and close on it

4

Run the bench on Core Ultra Series 2/3

one unchanged run turns NOT_THE_REQUIRED_MEASUREMENT into the measurement the track asks for

5

Publish the benchmark versioned

seeds, scorer and controls pinned, so a number reported against it stays comparable

Ordered by what the measurements say is blocking, not by what is easiest to demonstrate.

Check every number in this deck

github.com/jianwang-ntu/bimanual-dinner-table-so101 · public, MIT-licensed

`python3 scripts/verify_scene.py`

16 / 16 structural and physical checks

`python3 scripts/test_task_predicates.py`

11 / 11 scorer accept and reject controls

`python3 scripts/test_perception_pipeline.py`

18 / 18 perception and OpenVINO controls

`python3 scripts/eval_seeds.py`

regenerates the 19 / 50 and the 0 / 50 no-policy control

`python3 scripts/test_slides.py`

re-derives every figure on these slides and fails if one has gone stale

This deck is generated by `scripts/make_slides.py`, which reads `evidence/*.json`. No figure on any slide was typed by hand.